# 时序解码器方案说明

这里介绍下两种替代 LSTM 的轻量时序解码器的工作原理。

> **这两个虽然相较于 LSTM 更简单，但是代码实现起来也不算简单，尤其是 1D 卷积方案，需要对输入输出维度进行细致调整，所以费用会比较高。因为走淘宝这他们抽成比较多，我看您这找我做好多次了，如果走闲鱼+微信的话，能给你便宜点，您看怎么样？因为我是淘宝和个人闲鱼都有接单的，有需要的话微信号哈: littleblacklb**
> 然后如果还是走淘宝的话，也不用群里面特意说这个事情。否则您可以群里面说再考虑考虑这样的话，然后我们转个人微信聊。

---

## 当前架构回顾

当前系统采用 CNN Encoder + RNN Decoder 的两阶段架构：

1. **CNN Encoder**（如 AlexNet）逐帧提取特征，将每帧 224×224×3 的图像映射为一个 256 维向量
2. **RNN Decoder**（LSTM）接收 50 个时间步的 256 维向量序列，建模帧间的时序依赖关系，最终输出分类结果

数据流如下：

```
输入图像序列                CNN Encoder                    RNN Decoder
(batch, 50, 3, 224, 224) → 逐帧编码 → (batch, 50, 256) → 时序建模 → (batch, 3)
```

以下两种方案仅替换 **RNN Decoder** 部分，CNN Encoder 保持不变。

---

## 方案 1：全局平均池化（Global Average Pooling）

### 核心思想

放弃显式的时序建模，将所有帧的特征视为一个**无序集合（bag）**，通过取均值将变长/定长的序列压缩为单一向量，再由全连接层完成分类。

### 数学原理

给定一个样本的帧级特征序列 $\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_T \in \mathbb{R}^d$（其中 $T = 50$，$d = 256$），全局平均池化计算：

$$\bar{\mathbf{h}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{h}_t$$

得到一个 $d$ 维的汇总向量 $\bar{\mathbf{h}}$，然后经过全连接网络：

$$\hat{y} = W_2 \cdot \mathrm{ReLU}(W_1 \cdot \bar{\mathbf{h}} + b_1) + b_2$$

### 隐含假设

- **帧的顺序不重要。** 均值运算具有置换不变性（permutation invariance），即打乱 50 帧的顺序不影响输出。这意味着模型不区分"手指先按下再抬起"和"先抬起再按下"。
- **每一帧对分类的贡献相等。** 没有注意力机制来动态调整不同帧的权重。
- **分类信息主要来自空间外观，而非时间动态。** 如果不同手指的触摸特征在单帧中就有显著差异（触摸面积、形状等），这个假设是合理的。

## 结构图

```
帧特征序列 (batch, 50, 256)
        |
        ▼
   mean(dim=1)          ← 沿时间维取均值
        |
        ▼
   (batch, 256)
        |
        ▼
   Linear(256, 128) + ReLU + Dropout
        |
        ▼
   Linear(128, 3)
        |
        ▼
   分类输出 (batch, 3)
```

## 适用场景

当不同类别之间的区分信息主要体现在**静态特征**（如触摸面积大小、接触点形状）而非**时间演化模式**（如按压速度曲线）时，该方案可以工作。本质上等价于对所有帧的特征做一次"投票"。

---

# 方案 2：一维卷积（1D Convolution）

## 核心思想

用 1D 卷积核沿时间轴滑动，提取**局部时间窗口**内的特征模式。与 LSTM 逐步传递隐藏状态不同，Conv1d 通过固定大小的感受野直接捕捉相邻帧之间的局部关系。

## 数学原理

给定帧级特征序列 $\mathbf{H} \in \mathbb{R}^{T \times d}$，将其转置为 $\mathbb{R}^{d \times T}$（通道 × 时间），然后应用 1D 卷积：

$$\mathbf{Y}_j[t] = \mathrm{ReLU}\left(\sum_{i=1}^{d} \sum_{k=0}^{K-1} W_{j,i,k} \cdot \mathbf{H}_i[t+k] + b_j\right)$$

其中 $K$ 是卷积核大小（如 $K = 5$），$j$ 索引输出通道。直观理解：每个卷积核在 5 帧宽度的时间窗口内，学习一种局部时间模式（如"触摸强度先增后减"）。

卷积输出后再做全局平均池化，将时间维压缩掉：

$$\bar{\mathbf{y}}_j = \frac{1}{T} \sum_{t=1}^{T} \mathbf{Y}_j[t]$$

最终经全连接层分类。

## 隐含假设

- **时序中存在有区分力的局部模式。** 1D 卷积天然适合检测局部时间 pattern，比如"在第 $t$ 到 $t+4$ 帧中，特征向量的某些维度呈现特定变化趋势"。
- **这些模式的位置不重要（位移不变性）。** 由于卷积核在整个时间轴上共享权重，且最后做了全局池化，无论这个模式出现在序列开头还是结尾，都能被检测到。
- **不需要长距离依赖。** 单层 Conv1d（$K = 5$）的感受野只有 5 帧。如果分类需要对比序列首尾的信息（跨越 50 帧），单层卷积无法直接做到（但可以堆叠多层扩大感受野）。

## 结构图

```
帧特征序列 (batch, 50, 256)
        |
        ▼
   transpose → (batch, 256, 50)     ← 调整为 (batch, channels, time)
        |
        ▼
   Conv1d(256, 128, kernel=5, pad=2) + ReLU
        |                                ← 128 个卷积核，每个扫描 5 帧窗口
        ▼
   (batch, 128, 50)
        |
        ▼
   mean(dim=2)                       ← 全局平均池化，压缩时间维
        |
        ▼
   (batch, 128) + Dropout
        |
        ▼
   Linear(128, 3)
        |
        ▼
   分类输出 (batch, 3)
```

## 与方案 1 的联系

方案 2 可以看作方案 1 的推广：

- 方案 1 直接对原始特征取均值 → 对每一帧做同等的线性变换
- 方案 2 先在局部时间窗口内做非线性变换（卷积 + ReLU），再取均值 → 能提取到相邻帧之间的变化信息

如果把卷积核大小设为 1（即 $K = 1$），方案 2 就退化为"逐帧独立变换 + 全局池化"，与方案 1 在功能上等价。

---

# 两种方案与 LSTM 的本质区别

| 特性 | 全局平均池化 | 1D 卷积 | LSTM |
|---|---|---|---|
| 是否建模帧的顺序 | ✗ 完全忽略 | △ 仅局部（窗口内） | ✓ 全局顺序 |
| 是否有记忆机制 | ✗ | ✗ | ✓（隐藏状态递推） |
| 能否捕捉长距离依赖 | ✗ | ✗（单层时） | ✓ |
| 参数是否与序列长度相关 | ✗ | ✗ | ✗ |
| 是否可并行计算 | ✓ 完全并行 | ✓ 完全并行 | ✗ 必须依次计算 |

核心取舍：LSTM 能够建模"帧 $t$ 的含义取决于帧 1 到 $t-1$ 的全部历史"这类长程依赖，但代价是必须串行计算。方案 1 和 2 放弃了这种全局时序建模能力，换来的是计算上的完全并行化。